



Nagykanizsai SZC
Zsigmondy Vilmos Technikum

Vizsgaremek

Készítette: Cséfalvay Bálint, Horváth Márk Attila

2026.

Nagykanizsai SZC
Zsigmondy Vilmos Technikum
SZOFTVERFEJLESZTŐ- ÉS TESZTELŐ SZAK

M&B Lakhatás



GitHub link:

https://github.com/Markusz300/Alberlet_project-final

Tartalom

1. Bevezető.....	3
2. A Tervezés során használt szoftverek.....	4
2.1. Trello	4
2.2. Wireframe.cc	5
3. Fejlesztői dokumentáció.....	7
6.1.1. Hardver	11
6.1.2. Szoftverek	11
6.2. Az adatbázis bemutatása	15
6.3. Bankend és API dokumentáció	16
7. Felhasználói dokumentáció	23
7.1. Kezdőoldal	23
7.2. Hirdetések megtekintése	24
7.3. Hirdetés feladása	36
8. Tesztelés	43
9. Fejlesztési lehetőségek	45
Irodalomjegyzék.....	46

1. Bevezető

A vizsgaremekünk ötlete onnan indult, hogy szembesültünk már az albérletkeresés nehézségeivel: a legtöbb hirdetési oldal átláthatatlan és nehezen kezelhető. Elhatároztuk, hogy készítünk egy letisztult, modern platformot, amely nemcsak esztétikus, de a hirdetésfeladás és a keresés folyamatát is végtelenül egyszerűvé és felhasználóbaráttá teszi.

A digitális világ folyamatos fejlődésével egyre nagyobb igény mutatkozik az átlátható és könnyen használható online piacterek iránt. Arra gondoltunk, létrehozunk egy olyan webalkalmazást, amelyen magas szintű vizuális élményt nyújt, mégis egyszerű, könnyen kezelhető, és a következő funkciókat biztosítja:

- albérletek listázása, vizuálisan szép megjelenéssel,
- kártya és táblázatos nézet kombinációja a felhasználói élmény növelésére,
- hirdetés feladásának funkciója, amely egy űrlap kitöltésével, néhány lépésben egy új hirdetés feladására ad lehetőséget.

A technológia kiválasztásánál **Vue.js** és **Quasar Framework** mellett döntöttünk, mivel ezek modern, rugalmas és gyors fejlesztést tesznek lehetővé. A **Quasar** különösen nagy előnyt jelentett a reszponzív **UI-komponensei** miatt, hiszen így az oldalt egyszerűbben optimalizálhattuk asztali, mobil és tablet nézetre is.

A **backend** kiszolgálására a **Laravel** keretrendszert választottuk, amely az **Eloquent ORM** segítségével biztosítja az adatbázis-műveletek hatékony és objektumorientált kezelését. Az **Eloquent** használatával tiszta és könnyen karbantartható kódbázist hoztunk létre, ahol a bonyolult lekérdezések helyett modelleken keresztül kommunikálunk az adatokkal, így biztosítva a gyors fejlesztést és a magas szintű adatbiztonságot. A **Laravel** robusztus **API**-támogatása révén a szerveroldal zökkenőmentesen és biztonságosan szolgálja ki a **Quasar**-alapú **frontend** kéréseit, egységes és skálázható architektúrát alkotva a teljes alkalmazás alatt.

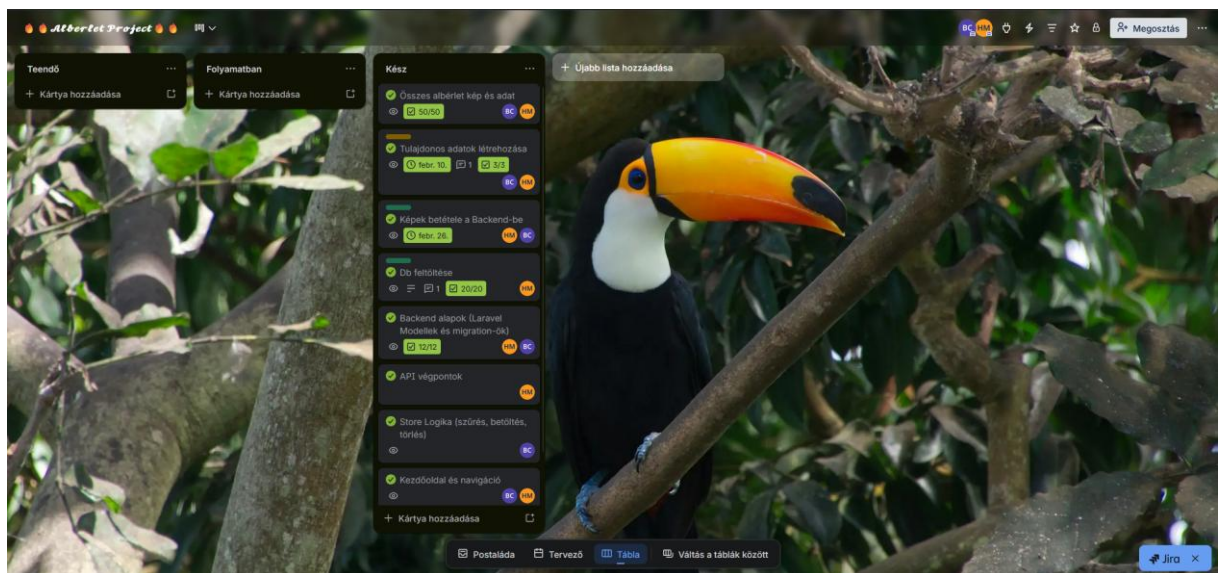
2. A Tervezés során használt szoftverek

2.1.Trello



A projektmenedzsmenthez a **Trello** webalapú alkalmazást használtuk, amely a Kanban-módszer segítségével tette lehetővé a fejlesztési folyamatok hatékony vizualizálását. A felületen kártyák és listák segítségével rendszereztük a feladatokat, így a munkafolyamat minden szakasza átláthatóvá és könnyen nyomon követhetővé vált.

A keretrendszer segítségével minden feladatot külön kártyára rögzítettünk, amelyekhez határidőket rendeltünk, megjegyzéseket fűztünk és a szükséges fájlokat is csatoltuk. A struktúra gerincét a "Teendő", "Folyamatban" és "Kész" listák alkották, ami nagy segítséget jelentett a fejlesztési prioritások kezelésében és a határidők pontos betartásában.



2.2. Wireframe.cc



A weboldal felületének megtervezéséhez a **Wireframe.cc** eszközt választottuk, mivel rendkívül egyszerű felülettel rendelkezik, és regisztráció nélkül is lehetővé teszi a haladást. A szoftver minimalizmusa segített abban, hogy a dekoráció helyett a tiszta funkcionalitásra és az elrendezésre koncentrálhassunk, így gyorsan elkészíthettük a projekt teljes tervrajzát.

A munkafolyamat során először az alapstruktúrát rögzítettük, kijelölve a fejléc, a menüsor, a fő tartalom és a lábléc helyét. Ezt követte a finomhangolás fázisa, amely során beállítottuk a pontos méreteket és arányokat, biztosítva, hogy a frontend fejlesztés már egy kész, logikusan felépített váz mentén indulhasson el. Ez a tervezés garantálja, hogy az **API**-ból érkező adatok pontosan a számukra kijelölt, ergonomikus helyre kerüljenek a felhasználói felületen.

A drótváz 3 méretben készült el:

- mobil nézet
- tablet nézet
- desktop nézet

Az elkészült drótvázak:

- Az oldal kezdőlapja:



- Az Albérlet kereső oldal:



- Az Albérlet Részletei oldal



3. Fejlesztői környezet üzembe helyezése

A projekt inicializálásához első lépésként futtassa az **npm install** parancsot a gyökérkönyvtárban, amely a **package.json** fájl alapján letölti és telepíti a futtatáshoz szükséges összes függőséget a **node_modules** mappába. Amennyiben a projekt architektúrája megköveteli a **Pinia** állapotkezelő használatát, de az még nem szerepel a függőségek között, az **npm install pinia** utasítással adható hozzá a rendszerhez. A telepítési folyamat végeztével javasolt az **npm audit fix** parancs végrehajtása, amely automatikusan orvosolja az ismert biztonsági sebezhetőségeket és frissíti az elavult csomagokat a kompatibilitási határokon belül. A fejlesztői környezet végső elindításához és a hot-reload funkcióval ellátott helyi szerver futtatásához az **npm run dev** parancs használatos, melyet követően az alkalmazás a terminálban megadott lokális címen válik elérhetővé tesztelésre és további fejlesztésre.

3.1 Fejlesztői dokumentáció

A **Pinia store**-ok alkotják az alkalmazás logikai központját. Itt történik az adatok feldolgozása, az API-val való kommunikáció és az állapotok globális kezelése. Az alábbiakban részletezzük az összes funkciót (Action-t) és azok feladatát.

Alberlet Store (useAlberletStore)

Ez a store felelős minden ingatlanhoz, helyszínhez és szűréshez kapcsolódó adatért.

Adatlekérdező funkciók

- **fetchMegyek(isFiltered)**: Lekéri a megyék listáját a backendről. Ha a paraméter igaz, csak azokat a megyéket adja vissza, ahol van aktív hirdetés.
- **fetchVarosok(isFiltered)**: Hasonlóan a megyékhez, a városok listáját tölti be, támogatva a hirdetés alapú szűrést.
- **fetchAlberletek(page, adminView)**: A legfontosabb lekérdező függvény. Összegyűjti az aktuális szűrőket, a lapozási adatokat, és lekéri a hirdetések listáját. Az adminView paraméterrel az inaktív hirdetések is elérhetővé válnak.
- **fetchAlberletById(id)**: Egy konkrét ingatlan összes adatát tölti be az egyedi azonosító alapján a részletes nézethez.
- **fetchAllForAdmin()**: Egy egyszerűsített hívás, amely kifejezetten az adminisztrációs felület számára kéri le az összes létező hirdetést.

Adatmódosító funkciók

- **saveAlberletUpdate(id, payload):** Meglévő hirdetés adatainak frissítésére szolgál. Sikeres mentés esetén pozitív visszajelzést (Notify) küld és frissíti a helyi listát.
- **updateStatus(id, status):** Kifejezetten a hirdetés aktiválására vagy kikapcsolására szolgál. Mivel a backend validációja szigorú, ez a függvény a státusz mellett az alapvető adatokat is elküldi a sikeres módosításhoz.
- **deleteAlberlet(id):** Véglegesen törli a hirdetést az adatbázisból és a szerverről, majd azonnal eltávolítja azt a frontend aktuális állapotából is.

Segédfunkciók

- **resetFilters():** Alaphelyzetbe állítja az összes szűrőfeltételt (ár, szoba, típus stb.) és újraindítja az alapértelmezett lekérdezést.
- **handleError(msg):** Egységesített hibaüzenet-kezelő, amely a Quasar Notify segítségével piros riasztást jelenít meg a felhasználónak.

Auth Store (useAuthStore)

- Ez a store kezeli a hozzáférési jogosultságokat és az adminisztrátori munkamenetet.
- **login(user, pass):** Ellenőrzi a megadott hitelesítő adatokat. Amennyiben egyeznek, az isLoggedIn állapotot igazra állítja, és a biztonság kedvéért a localStorage-ba is elmenti a státuszt, hogy az oldal frissítésekor ne jelentkezzen ki a rendszer.
- **logout():** Megszünteti az adminisztrátori munkamenetet. Törli az állapotot és eltávolítja a kulcsot a böngésző tárolójából, így a védett útvonalak (Guards) azonnal elérhetatlenné válnak.

Frontend InterFace-ek

Hirdetés Objektum (Alberlet Interfész)

- Ez az interfész írja le, hogy milyen adatokból áll egy ingatlanhirdetés, amikor megérkezik a backendről:
- **id:** Az ingatlan egyedi azonosítója.
- **leiras:** A hirdetés szöveges bemutatása.
- **ar:** Az albérlet havi bérleti díja.
- **meret:** Az ingatlan alapterülete négyzetméterben.
- **szobaszam:** A szobák száma.
- **aktiv:** Egy logikai érték (0 vagy 1), ami jelzi, hogy a hirdetés látható-e a nagyközönség számára.
- **butorozott** és **lift:** Igen/Nem jellegű tulajdonságok.
- **kepek:** Egy lista, ami a feltöltött fotók elérési útját tárolja.

Szűrő Objektum (Filters Interfész)

- Ez az interfész felelős azért, hogy a keresőfelületen beállított értékeket tárolja:
- **megye_id** és **varos_id:** A földrajzi szűréshez szükséges azonosítók.
- **min_ar** és **max_ar:** Az ár intervallum határai.
- **min_szoba** és **max_szoba:** A szobaszám tartománya.
- **min_meret** és **max_meret:** A négyzetméter alapú szűrés határai.
- **butorozott**, **lift**, **tipus:** Kategória alapú szűrések.
- **sort:** A lista rendezési elve (például legolcsóbb vagy legújabb elől).

Lapozási és Állapot Interfész (Pagination)

- Ez segít a felhasználónak navigálni a sok hirdetés között:
- **currentPage:** Megmutatja, melyik oldalon jár éppen a felhasználó.
- **totalPages:** A rendszer kiszámolja, összesen hány oldalnyi hirdetés van.
- **loading:** Egy jelzőfény a programnak: ha igaz, akkor éppen töltődnek az adatok, és érdemes egy várakozó ikont mutatni.

Képkezelő Interfész (Getter)

- Ez egy speciális átalakító, ami a nyers adatból készít megjeleníthető formátumot:
- Megvizsgálja, hogy van-e kép a hirdetéshez.
- Ha nincs, egy előre beállított alapértelmezett képet ad vissza.
- Ha van, összefűzi a szerver címét a kép nevével, hogy a böngésző meg tudja találni a fájlt.

Szűrő-tisztító Interfész (Getter)

- Ez a funkció készíti elő az adatokat az **API** hívás előtt:
- Végignézi a beállított szűrőket.
- Kidobja azokat, amik üresek (vagyis a felhasználó nem állított be ott semmit).
- Csak a tényleges keresési feltételeket küldi el a szervernek, hogy ne terhelje felesleges adatokkal.

Hitelesítési Interfész (Auth State)

- Az adminisztrátori belépést kezeli:
- **isLoggedIn**: Megmondja, hogy az illető sikeresen bejelentkezett-e.
- **isAdmin**: A böngésző tárolójában (localStorage) lévő jelzés, ami segít emlékezni a belépésre oldalfrissítés után is.

A frontend útvonalkezelését a **Vue Router** végzi, amely a böngésző címsorát szinkronizálja az éppen megjelenített tartalommal. Az útvonalakat két fő csoportra osztottuk: **publikus** és **védett** felületekre.

- **Publikus útvonalak** Ezeket az oldalakat minden látogató szabadon elérheti:
- **Kezdőlap**: Az alkalmazás alapértelmezett nyitóoldala.
- **Keresés**: Az albérletek listázására és a szűrőfeltételek beállítására szolgáló felület.
- **Hirdetésfeladás**: Egy űrlap, amelyen keresztül bárki beküldhet új ingatlanhirdetést.
- **Részletek**: Dinamikus útvonal, amely az ingatlan egyedi azonosítója alapján tölti be a specifikus adatlapot.
- **Védett útvonalak és biztonság**
- Az adminisztrációs felületekhez (statisztikák, hirdetések jóváhagyása vagy törlése) **csak bejelentkezés után** lehet hozzáférni.
- **Login oldal**: Itt történik a hitelesítés.
- **Admin felület**: A hirdetések kezelésének központja.
- **Szerkesztés**: Meglévő hirdetések módosítása.

3.1.1. Hardver

- A gép alkatrészei, amin folyt a tesztelés:
 - Processzor: Intel Core i5-10500 CPU @ 3.10GHz
 - Videókártya: Intel UHD Graphics 630
 - Memória mérete: 16 GB
 - Háttértár: WDC PC SN730 SDBQNTY-256G-1001
 - Monitor: Lenovo ThinkVision T24i-20 1920x1080p
 - Operációs rendszer verziója: Windows 11 Pro 25H2

3.1.2. Szoftverek

Visual Studio Code



A vizsgaremek **frontend és backend** rétegeit egyaránt **Visual Studio Code** környezetben fejlesztettük. Ez a Microsoft által gondozott, ingyenes és nyílt forráskódú kódszerkesztő program a sokoldalúságának, valamint számos beépített funkciójának köszönhetően hatékony támogatást nyújtott a teljes munkafolyamat során, a felület kialakításától egészen a kiszolgálóoldali logika megvalósításáig. Használt bővítmények:

- HTML, CSS Support
- Auto Close Tag, Auto Rename Tag: HTML támogatás
- Vetur, Vue Official: Vue.js támogatás
- Prettier – Automatikus kódformázás
- Quasar Docs
- json
- PHP, PHP Getters & Setters, PHP Profiler
- Node.js Exec
- JavaScript (ES6) code snippets
- Composer

XAMPP



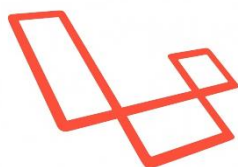
A **XAMPP** egy ingyenes szervercsomag, amely Apache webszervert és PHP-támogatást biztosít, így nélkülözhetetlen volt a helyi fejlesztői környezetünk kialakításához. Mivel a projektünk Laravel-alapú, a XAMPP segítségével futtattuk lokálisan a PHP-kódot és szolgáltuk ki a weboldalt a fejlesztés során. Bár a csomag tartalmazza a MariaDB-t is, az alkalmazásunkhoz tartozó adatbázist külső környezetben kezeltük, a XAMPP-ot pedig a szerveroldali logika és a webes kiszolgálás stabil háttereként alkalmaztuk.

DBeaver



Az adatbázis-struktúra kialakításához és a táblák kezeléséhez a **DBeaver** szoftvert alkalmaztuk. Ez egy univerzális, nyílt forráskódú adatbázis-menedzsment eszköz, amely vizuális felületével nagyban megkönnyítette számunkra az adatok rendszerezését, az SQL-lekérdezések futtatását, valamint az adatbázis és az alkalmazás közötti kapcsolat ellenőrzését.

Laravel



A **Laravel** egy nyílt forráskódú, PHP-alapú keretrendszer, amelyet a modern webalkalmazások fejlesztésére terveztünk. A projekt során azért döntöttünk mellette, mert az **MVC (Model-View-Controller)** szemléletmódot követi, ami lehetővé tette számunkra a kód tiszta tagolását és a fejlesztési feladatok hatékony megosztását.

Vue.js



A **Vue.js** egy progresszív JavaScript keretrendszer, amelyet Evan You hívott életre 2014-ben, és amelyet a projektünk frontend felületének fejlesztéséhez alkalmaztunk. A keretrendszer egyik legnagyobb előnye a virtuális DOM (Document Object Model) használata: ez a technológia lehetővé teszi, hogy a felületen történő változások rendkívül gyorsan és hatékonyan frissüljenek, biztosítva ezzel a kiváló teljesítményt és a gördülékeny felhasználói élményt.

A projekt a következő eszközkészleteket tartalmazza:

Vite



A **Vite** egy modern fejlesztői szerver (Local Development Server), amelyet a Vue-alapú projektünk kiszolgálásához használtunk. A szoftver folyamatosan monitorozza a fájlokat szerkesztés közben, és mentéskor a böngésző azonnal frissíti a módosított kódot a Hot Module Replacement (HMR) technológia segítségével. Ez a folyamat rendkívül hatékony, mivel csak az adott, megváltozott fájlt tölti be újra, ahelyett, hogy a teljes alkalmazást újrafordítaná, így jelentősen felgyorsította a fejlesztési munkánkat.

Quasar Framework



A **Quasar** egy Vue.js alapokra épülő keretrendszer, amelyet a modern webes felületek fejlesztésének megkönnyítésére és gyorsítására használtunk. Mivel a Quasar a Vue 3-at és a Composition API-t alkalmazza, lehetővé tette számunkra a tiszta és moduláris kódolást a teljes folyamat során. A keretrendszer számos előre gyártott UI komponenst tartalmaz, így például gombokat, menüket, táblázatokat és űrlapokat emeltünk be kész elemként, ami jelentősen felgyorsította a vizuális felületek felépítését. Nagy előnye a platformfüggetlenség, hiszen ugyanazzal a kódbázissal akár webes, mobil- vagy asztali alkalmazást is fejleszthetnénk. A beépített grid rendszernek köszönhetően könnyedén biztosítottuk a rezponzív megjelenést minden kijelzőméretre, miközben a Layout Builder segítségével alakítottuk ki az oldal szerkezetét a navigációs sávoktól a fejlécgig. A fejlesztést ezen felül a keretrendszer saját beépített pluginjai is segítették, amelyekkel a funkcionális feladatokat és az értesítéseket egyszerűen tudtuk kezelni.

Vue Router



A **Vue Router** a Vue.js hivatalos útvonalválasztó megoldása, amelyet az egyoldalas alkalmazásunk (SPA) navigációjának kezeléséhez használtunk. Segítségével lehetővé tettük a különböző komponensek dinamikus megjelenítését az URL alapján, így a projektünkben az oldalak közötti váltás az oldal tényleges újratöltése nélkül történik meg. A fejlesztés során a QLayout komponenst alkalmaztuk, amelynek köszönhetően az egyes nézetek ugyanazon a fejlécen és navigációs menün osztoznak, biztosítva ezzel a folyamatos és gyors felhasználói élményt a böngészés során.

Axios



Az **Axios** egy modern HTTP kliens, amely a Node.js környezetben és a böngészőben egyaránt hatékonyan kezeli a backend szerverrel való adatcserét. A projektünk során a HTTP kérések küldésére, valamint az adatok

lekérésére és továbbítására alkalmaztuk, a folyamatokat pedig aszinkron módon, Promise-okon keresztül menedzseljük. A fejlesztés során egy Axios Instance-t, vagyis egy előre konfigurált példányt hoztunk létre, aminek segítségével elkerültük a konfigurációs beállítások folyamatos ismétlését. Ez az előre meghatározott példány biztosította, hogy minden egyes kérésünk automatikusan ugyanazokat az alapbeállításokat használja, ami tisztább kódszerkezetet és stabilabb kommunikációt eredményezett a frontend és a backend rétegek között.

Pinia Store



A **Pinia** a Vue.js hivatalos állapottároló könyvtára, amelyet azért alkalmaztunk, hogy az alkalmazás különböző oldalai és komponensei között hatékonyan meg tudjuk osztani az adatokat. Mivel ez a megoldás teljes mértékben kompatibilis a Vue Composition API-val, a globális állapotot jól strukturált store-okba szerveztük, amelyek különálló állapotokat, akciómetódusokat és gettereket tartalmaznak. A Pinia segítségével kezeltük az alkalmazás központi adatait, és ezen keresztül

valósítottuk meg az aszinkron kommunikációt is a Laravel backenddel. A fejlesztés során a store-okon belül két fő típusú akciómetódust használtunk: az egyiket az adatok lekérésére (get), a másikat pedig az adatok módosítására és az új rekordok létrehozására (post) vonatkozó műveletek végrehajtására.

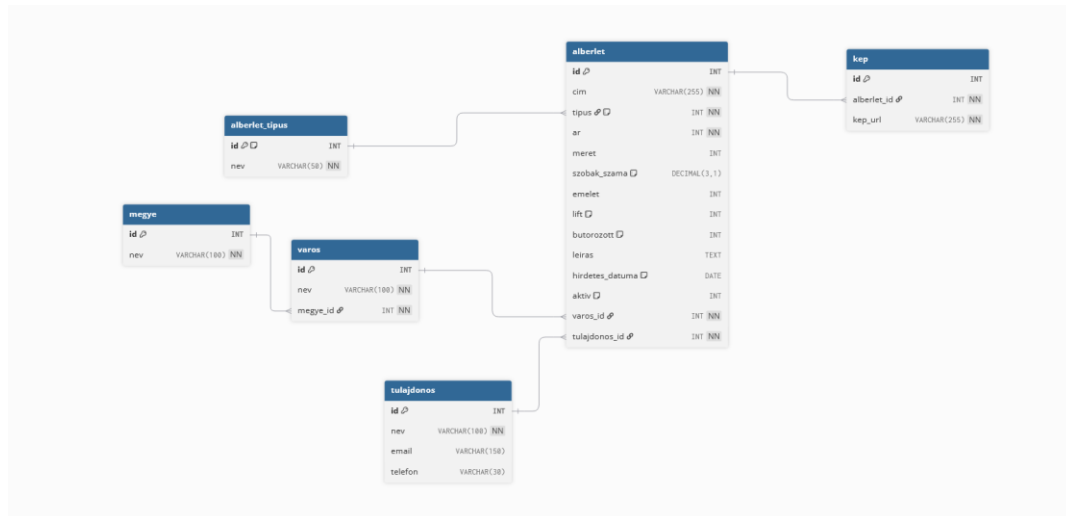
GitHub Desktop



A **GitHub Desktop** alkalmazást a Git verziókezelő rendszer használatának megkönnyítésére és vizuális támogatására alkalmaztuk. Ez a grafikus felhasználói felület lehetővé tette számunkra, hogy a parancssori műveletek helyett egy átlátható ablakban kezeljük a projektünk változtatásait, így a

kódok commitolása és a távoli repository-ba való feltöltése sokkal gördülékenyebbé vált. Az alkalmazás segítségével könnyen áttekinthettük a fájlokban végrehajtott módosításokat, kezeltük a különböző ágakat, és szinkronizáltuk a munkánkat a GitHub szerverével, ami jelentősen csökkentette a hibázási lehetőségeket és felgyorsította a kettőnk közötti közös fejlesztési folyamatot.

3.2. Az adatbázis bemutatása



- **(Megye):** Ez a tábla tárolja a megyék neveit (pl. Zala, Pest). Ez a földrajzi hierarchia legfelső szintje az adatbázisban, alapjául szolgál a városok besorolásának.
- **(Város):** A települések neveit tartalmazza. Minden várost egy idegen kulcson (megye_id) keresztül hozzáköt egy konkrét megyéhez, így tudjuk, hogy az adott város melyik megyében található.
- **(Tulajdonos):** Az ingatlanokat bérbeadó személyek kapcsolattartási adatait (név, e-mail cím, telefonszám) rögzíti. Egy tulajdonoshoz később több albérlet is tartozhat.
- **(Albérlet típus):** Fixen meghatározza az ingatlanok kategóriáit (0 = ház, 1 = lakás, 2 = szoba), hogy az adatbázisban egységes legyen az elnevezés és elkerülhetőek legyenek az elgépelések.
- **(Albérlet): Ez a rendszer központi táblája.** Itt tárolják a konkrét kiadó ingatlanok minden lényegi tulajdonságát:
 - Alapadatok (cím, ár, méret négyzetméterben, szobák száma).
 - Jellemzők (emelet, van-e lift, bútorozott-e).
 - Adminisztratív adatok (leírás, hirdetés feladásának dátuma, és hogy aktív-e még a hirdetés).
 - *Kapcsolatok:* Ez a tábla köti össze az ingatlant a helyszínnel (varos_id), a bérbeadóval (tulajdonos_id) és az ingatlan kategóriájával (tipus).

- **(Kép):** Az albérletekhez feltöltött fényképek elérési útvonalait (`kep_url`) tárolja (pl. a szerveren lévő mappa elérhetőségeit). Minden kép egy konkrét hirdetéshez (`alberlet_id`) van rendelve. Mivel ez egy külön tábla, egyetlen albérlethez így tetszőleges számú kép tartozhat (egy-a-többhöz kapcsolat).

3.3. Backend és API dokumentáció

Backend elindítása

A backend szolgáltatások konfigurálása a függőségek feloldásával kezdődik, melyhez a **composer install** parancs futtatása szükséges; ez a művelet a **composer.json** fájl alapján telepíti a keretrendszerhez és az alkalmazáshoz szükséges PHP csomagokat. Ezt követően a médiafájlok és feltöltések megfelelő kezelése érdekében létre kell hozni a szimbolikus linket a publikus könyvtár és a tároló mappa között a **php artisan storage:link** utasítással. Az adatbázis-struktúra inicializálása vagy frissítése a **php artisan migrate** paranccsal történik, amely lefuttatja a migrációs fájlokat, létrehozva a szükséges táblákat és mezőket a konfigurált adatbázisban. A fejlesztői munka megkezdéséhez végül a **php artisan serve** parancs használatával indítható el a PHP beépített fejlesztői szerver, amely alapértelmezetten a **http://127.0.0.1:8000** címen teszi elérhetővé az API végpontokat és a backend szolgáltatásokat.

Ez az **API** egy ingatlanhirdetési rendszer backend kiszolgálója, amely lehetővé teszi albérletek (házak, lakások, szobák) kezelését, böngészését és szűrését. A rendszer központi funkciója a hirdetések közzététele, ahol a tulajdonosok adatai (e-mail, név, telefon) automatikusan szinkronizálódnak, elkerülve az adatok duplikálását vagy a hibás azonosítást. Az **API RESTful** elveket követ, **JSON formátumban** kommunikál, és támogatja a komplex szűrési feltételeket, a többképes feltöltést, valamint a földrajzi alapú (város/megye) kategorizálást.

Controllerek bemutatása:

1. AlberletController

Ez a projekt központi eleme, amely az összes hirdetéssel kapcsolatos műveletet (**CRUD**) irányítja.

- **index (Keresés és listázás):**
 - **Intelligens szűrés:** Kezeli a megye, város, ár, méret és szobaszám szerinti szűréseket.
 - **Felszereltség:** Képes szűrni típusra (lakás/ház/szoba), bútorozottságra és liftre.
 - **Admin nézet:** Az **admin_view** paraméterrel szabályozza, hogy csak az aktív hirdetések vagy az összes rekord jelenjen meg.
- **store (Hirdetésfeladás):**
 - **Tranzakciókezelés:** DB::transaction-t használ, így hiba esetén nem maradnak félkész adatok az adatbázisban.
 - **Automatizmus:** Ha a felhasználó új várost vagy megyét ad meg, a rendszer automatikusan létrehozza azokat.
 - **Címformázás:** Egy beépített algoritmus egységesíti a címeket (nagy kezdőbetűk, írásjelek).
- **update (Módosítás és státusz):**
 - Itt történik a hirdetések aktiválása az aktiv mező módosításával.
 - **Biztonság:** Ellenőrzi, hogy ha egy létező e-mail címet használnak, a név és a telefon egyezzen a korábban tárolttal.
- **destroy (Törlés):**
 - A hirdetés törlése mellett fizikailag is eltávolítja a csatolt képeket a szerver tárhelyéről.

2. TulajdonosController

Ez a vezérlő a hirdetőik adatainak kezelését segíti.

- **check (Gyorsellenőrzés):** E-mail cím alapján ellenőrzi, hogy a hirdető szerepel-e már az adatbázisban. Ha igen, visszaküldi a nevet és telefonszámot az űrlap automatikus kitöltéséhez.

3. MegyeController & VarosController

Ezek a vezérlők szolgálják ki a helyszínadatokat a keresőhöz és az űrlapokhoz.

- **index:** Listázza az összes megyét és várost.
- **Filtered paraméter:** Ha a filtered=true aktív, csak azokat a helyszíneket adja vissza, amelyekhez tartozik legalább egy aktív hirdetés.

Model osztályok

Az adatbázis tábláit a reprezentáló osztályok Laravel-ben a Model osztályok. Az adatbázisműveletekhez az Eloquent ORM megoldásait használtuk. Az adatbázis tábláit projektünkben a következő osztályok képviselik:

Tábla az adatbázisban	Laravel osztály
alberlet	Alberlet
alberlet_tipus	AlberletTipus
kep	Kep
megye	Megye
varos	Varos
tulajdonos	Tulajdonos

Resource osztályok

A Resource osztályok felelősek a JSON-válaszok megfelelő formátumának meghatározásáért.

A következő erőforrás-osztályt használtuk:

- AlberletResource

API végpontok:

index:

GET

`/api/alberletek`

Listázza az összes hirdetést a szűrők alapján.

show:

GET

`/api/alberletek/{id}`

Megjeleníti egy konkrét hirdetés összes adatát.

store:

POST

/api/alberletek

Elmenti az új hirdetést, a tulajdonost és a képeket.

Content-Type: multipart/form-data (mivel fájlokat is kell küldeni)

Változó neve	Típus	Kötelező-e	Leírás
cim	String	I	Max 255 karakter. A backend automatikusan formázza.
tipus	Integer	I	0: ház, 1: lakás, 2: szoba.
ar	Integer	I	Minimum 10 000 .
meret	Integer	I	Minimum 5 (m ²).
szobak_szama	Number	I	Minimum 0.5 . (Ház esetén automatikusan 1 lesz).
leiras	String	I	Minimum 20 karakter . A HTML tageket a szerver törli.
varos_id	Mixed	I	Meglévő város ID-ja VAGY az új város neve.
nev	String	I	Hirdető neve (3-100 karakter).
email	String	I	Valid email formátum.
telefon	String	I	Minimum 10 karakter.
kepek[]	File	I	Képfájlok (jpeg, png, jpg). Max 5MB/kép.
emelet	Integer	N	Alapértelmezett: 0 .
lift	Boolean	N	true / 1 vagy false / 0 .
butorozott	Boolean	N	true / 1 vagy false / 0 .
megye_id	Mixed	N	Csak ha új várost adsz meg: Megye ID-ja vagy neve.

update:

PUT

/api/alberletek/{id}

Módosítja a hirdetést vagy átállítja az állapotát (aktiválás).

Változó neve	Típus	Kötelező-e	Leírás
leiras	String	I	Az ingatlan leírása (min. 20 karakter).
ar	Number	I	A bérleti díj összege (min. 0).
cím	String	N	Az ingatlan címe.
meret	Integer	N	Alapterület (m ²).
szobak_szama	Number	N	Szobák száma.
emelet	Integer	N	Hányadik emeleten található.
aktiv	Boolean	N	A hirdetés státusza (látható-e a listában).
lift	Boolean	N	Lift megléte (értékei: "van", true, 1).
butorozott	Boolean	N	Bútorozottság (értékei: "igen", true, 1).
tipus	Integer	N	0: ház, 1: lakás, 2: szoba.
varos	Mixed	N	Város azonosítója vagy új város neve.
megye	Mixed	N	Megye azonosítója vagy neve (új városnál).
tulajdonos_neve	String	N	A hirdető nevének módosítása.
tulajdonos_tel	String	N	A hirdető telefonszámának módosítása.
tulajdonos_email	String	N	A hirdető e-mail címének módosítása.

destroy:

DELETE

/api/alberletek/{id}

Törli a hirdetést és a hozzá tartozó fájlokat.

index (Varos):

GET

/api/varosok

Visszaadja a városokat (akár hirdetés szerint szűrve).

index (Megye):

GET

/api/megyek

Visszaadja a megyéket (akár hirdetés szerint szűrve).

check:

GET

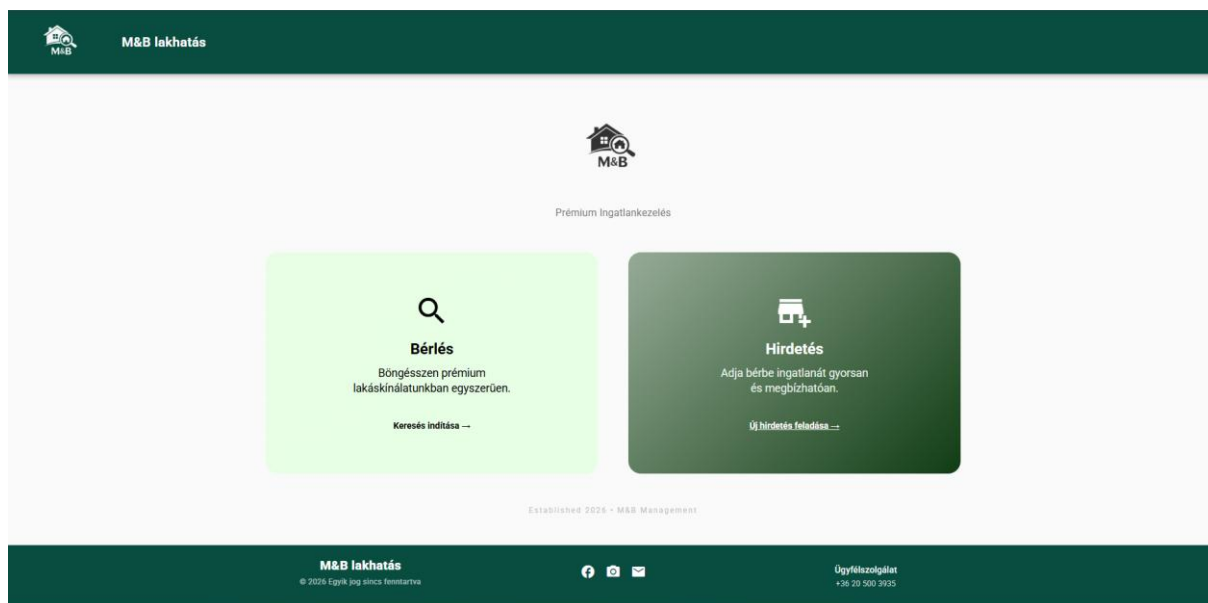
/api/users/check

Megnézi e-mail alapján, létezik-e már a hirdető.

4. Felhasználói dokumentáció

Ez az útmutató segít eligazodni a M&B lakhatás weboldalán. A weboldal albérlet keresésre és hirdetés feladására alkalmas.

4.1. Kezdőoldal



- **Bérlés kártya:** Erre kattintva a felhasználó a hirdetésböngésző oldalra jut, ahol megtekintheti az aktuálisan elérhető lakáskínálatot.
- **Hirdetés kártya:** Ez a gomb a hirdetésfeladás felületére navigál, ahol a tulajdonosok rögzíthetik kiadó ingatlanuk adatait.
- Az oldal alján található láblécben kaptak helyet a közösségi média elérések ezek minden oldal alján megtalálhatóak és a közvetlen kapcsolatfelvételi lehetőségek:
 - **Facebook & Instagram:** Az ikonok a készítő hivatalos közösségi oldalaira mutatnak.
 - **Email:** A levél ikonra kattintva a rendszer automatikusan megnyitja a levelezőprogramot a megadott e-mail címmel.
 - **Ügyfélszolgálat:** A telefonszám interaktív, így rákattintva (főleg mobileszközről) azonnal hívást kezdeményezhet a felhasználó.

4.2. Hirdetések megtekintése

The screenshot shows the 'M&B lakhatás' website. At the top, there's a dark green header with the logo and name. Below it, a section titled 'Albérlet Kereső' (Apartment Search) with the tagline 'Találd meg álmaid otthonát'. A green button labeled 'ÖSSZES SZŰRŐ' (All Filters) is in the top right. The main area displays a grid of apartment listings. Each listing includes a photo, address, price, and basic details like room count and area. The first row shows four listings: 9400 Sopron (2.5 szoba, 58 m², 1. em, 219 000 Ft), 6721 Szeged (2.0 szoba, 50 m², 2. em, 235 000 Ft), 6720 Szeged (1.0 szoba, 40 m², 2. em, 200 000 Ft), and 9400 Sopron (2.5 szoba, 42 m², 2. em, 190 000 Ft). Each listing has a 'RÉSZLETEK' (Details) link. The second row shows four more listings, partially visible.

- **Visszatérés a kezdőlapra:** Az oldal bal felső sarkában található logó, valamint az „M&B lakhatás” felirat interaktív. Bármelyikre kattintva a felhasználó azonnal visszajut a főoldalra, így a navigáció egyszerű és következetes.
- **Összes szűrő gomb:** A jobb felső sarokban elhelyezett zöld gomb segítségével érhető el a részletes szűrőpanel.
- **Személyre szabás:** A gomb megnyit egy felületet, ahol a felhasználó saját igényei szerint (például ár, alapterület vagy szobaszám alapján) szűkítheti a találati listát, hogy csak a számára releváns hirdetések jelenjenek meg.

Ár (Ft/hó)	m Szobák	Méret (m²)	Ingatlan típusa
Min Max	Min Max	Min Max	Összes típus
Város	Megye	Kényelem	Rendezés
Város választása	Megye választása	Bérel? Mindegy	Legújabb elől
KERESÉS INDÍTÁSA			SZŰRŐK TÖRLÉSE

- A keresési találatok pontosítása érdekében a felhasználó meghatározhatja az ingatlan havi bérleti díjának tartományát az **Ár (Ft/hó)** szekcióban.
- **Értékhatárok megadása:** A felületen két külön beviteli mező található:
 - **Min:** Itt adható meg a bérleti díj alsó határa.
 - **Max:** Itt adható meg az a maximális összeg, amelyet a felhasználó az albérletre szán.
- **Adatellenőrzés (Validáció):** A rendszer ügyel a helyes adatbevitelre. Amennyiben a felhasználó negatív számot (mínusz értéket) próbál beírni, a program hibaüzenettel jelzi, hogy a bevétel érvénytelen. Ez biztosítja, hogy csak reális keresési feltételek fussanak le a rendszerben.

Ár (Ft/hó)

Min: 50000, Max: 100000

Nem lehet negatív érték!

- **Szobák száma (Min/Max):** A felhasználó megadhat egy alsó és egy felső korlátot a kívánt szobaszámra vonatkozóan.
- **Tört értékek kezelése:** A beviteli mezők elfogadják az egész és a fél szobákat is (például: 1; 1.5; 2; 2.5 stb.), így a keresés pontosan igazodik a valós kínálatához.
- **Hibakezelés:** A mezők csak pozitív számokat fogadnak el.
 - Amennyiben a felhasználó negatív értéket ad meg, a rendszer **vizuális figyelmeztetéssel** (piros keret és felkiáltójel ikon), valamint egy hibaüzenettel („Nem lehet negatív érték!”) jelzi a hibát. Ez megakadályozza a logikátlan keresési feltételek beküldését.

Szobák

Min: 2,5, Max: 3

Nincs negatív szoba!!

- A keresett ingatlan méretét a felhasználó a **Méret (m²)** szekcióban szűkítheti le, így biztosítva, hogy csak az igényeinek megfelelő életterű találatok jelenjenek meg.
- **Területi tartomány megadása:** A szűréshez megadható az ingatlan alapterületének minimum (**Min**) és maximum (**Max**) értéke. Az egység meghatározása négyzetméterben (m²) történik.
- **Precíz adatbevitel:** A mezők támogatják a nem egész számok (tizedes értékek) megadását is (például: 45,6 m²), ami lehetővé teszi a pontosabb keresési feltételek beállítását.
- **Hibakezelés:** A rendszer valós időben ellenőrzi a bevitt karaktereket.
 - Amennyiben a felhasználó negatív értéket gépel be, a felület azonnal jelez: a beviteli mező kerete pirosra vált, egy felkiáltójel ikon jelenik meg, alatta pedig a „Nem lehet negatív érték!” figyelmeztetés olvasható. Ez a kontroll mechanizmus megakadályozza a hibás lekérdezések futtatását.

- A szűrőfelület ezen része lehetővé teszi, hogy a felhasználó az épület típusa alapján szűkítse a találati listát.
- **Választási lista:** Az **Ingatlan típusa** feliratra kattintva egy legördülő menü jelenik meg, amely rögzített opciókat kínál a hibátlan adatválasztás érdekében.
- **Kategóriák:** A felhasználó három fő típus közül választhat:
 - **Ház:** Önálló családi házak vagy ikerházak bérlete.
 - **Lakás:** Társasházi lakások vagy panellakások kínálata.
 - **Szoba:** Lakáson belüli külön bérelhető szobák vagy lakótársi lehetőségek.
- **Alapértelmezett állapot:** Alaphelyzetben az „Összes típus” opció aktív, amely az összes elérhető ingatlant megjeleníti, függetlenül azok kategóriájától.

- A felhasználó a **Város** szekcióban szűkítheti a keresést a kívánt földrajzi helyszínre. A rendszer rugalmas bevittet tesz lehetővé, hogy a böngészés minél kényelmesebb legyen.
- **Kétirányú bevitel:**
 - **Választás listából:** A mezőre kattintva egy legördülő menü jelenik meg, amely tartalmazza az összes olyan várost, ahol jelenleg elérhető albérlet.
 - **Szabadszöveges keresés:** A felhasználó manuálisan is elkezdheti begépelni a város nevét a mezőbe, így gyorsabban megtalálhatja a keresett települést.
- **Keresési visszajelzés:** Amennyiben olyan várost adunk meg, amelyben a rendszer jelenleg nem tart nyilván aktív hirdetést, az alkalmazás egy barátságos tájékoztató üzenettel jelzi ezt: „Sajnos még nincs itt albérletünk :(...”.

The image shows three examples of the 'Város' (City) dropdown menu in a web application:

- Left screenshot:** The dropdown is open, showing a list of cities. 'Budapest' is highlighted. Other visible cities include Győr, Hódmezővásárhely, Keszthely, Nagykanizsa, Pécs, Sopron, Szeged, Szombathely, and Zalaegerszeg.
- Middle screenshot:** The dropdown is open, showing 'Nagy' as the selected city. 'Nagykanizsa' is visible below it.
- Right screenshot:** The dropdown is open, showing 'Debrecen' as the selected city. Below the list, a message reads: 'Sajnos még nincs itt albérletünk :(...'.

- A hatékonyabb keresés érdekében a felhasználók nemcsak településre, hanem tágabb földrajzi egységre, vármegyékre is szűrhetnek a **Megye** szekció használatával. Ez a funkció a városválasztóhoz hasonló logikára épül.
- **Kétirányú bevitel:**
 - **Választás listából:** A mezőre kattintva megjelenik egy legördülő menü, amely felsorolja azokat a vármegyéket, ahol az adatbázisban aktuálisan elérhető hirdetés található.
 - **Szabadszöveges keresés:** A felhasználónak lehetősége van a megye nevének begépelésére is, ami felgyorsítja a választási folyamatot.
- **Keresési visszajelzés:** Ha a felhasználó olyan megyét gépel be, amely nem szerepel a kínálatban, vagy hibásan adja meg a nevet, a rendszer a „Nincs ilyen megye...” hibaüzenettel tájékoztatja az eredménytelenségről.

The image displays three examples of the 'Megye' (County) selection interface:

- Top Left:** A dropdown menu titled 'Megye' with the placeholder 'Megye választása'. The list of counties shown includes Zala, Győr-moson-sopron, Baranya, Vas, Pest, and Csongrád.
- Top Right:** A search bar titled 'Megye' with the placeholder 'Megye keresése...'. The text 'Za' is entered, and the suggestion 'Zala' is displayed below.
- Bottom:** A search bar titled 'Megye' with the placeholder 'Megye keresése...'. The text 'Fejér' is entered, and the message 'Nincs ilyen megye...' (No such county...) is displayed below.

- A **Kényelem** szekció lehetővé teszi a felhasználók számára, hogy az ingatlan felszereltsége és az épület szolgáltatásai alapján finomítsák a keresést. Ez a rész két specifikus szempontot fed le: a bútorozottságot és a lift elérhetőségét.
- **Szűrési feltételek:**
 - **Bútorozottság (Bútor?):** Itt adható meg, hogy a bérlő berendezett vagy üres ingatlant keres-e.
 - **Lift (Lift?):** Ezzel a feltétellel szűrhetők azok a társasházak, amelyek rendelkeznek lifttel.
- **Választható opciók:** Mindkét beállítás egy legördülő menüből érhető el, amely három egyértelmű lehetőséget kínál:
 - **Igen:** Csak az adott jellemzővel rendelkező ingatlanok megjelenítése.
 - **Nem:** Az adott jellemzővel nem rendelkező ingatlanok megjelenítése.
 - **Mindegy (Alapértelmezett):** Ez a rendszer alapbeállítása. Ebben az esetben a szűrő inaktív, tehát a találati listában vegyesen szerepelnek a bútorozott/bútorozatlan, illetve a lifttel rendelkező/nem rendelkező ingatlanok is.

Kényelem

Bútor?
Mindegy

Lift?
Mindegy

Mindegy

Igen

Nem

- A találati lista sorrendjét a felhasználó a **Rendezés** opció segítségével szabhatja személyre. Ez a funkció segít abban, hogy a keresési eredmények a bérlő számára legfontosabb szempont (frissesség vagy ár) szerint jelenjenek meg.
- **Rendezési módok:** A legördülő menü három különböző logikai sorrendet kínál:
 - **Legújabb elől (Alapértelmezett):** A rendszer a hirdetés feladásának időpontja alapján rendezi az ingatlanokat. A legfrissebben feltöltött hirdetés kerül a lista elejére, biztosítva, hogy a felhasználó azonnal értesüljön az új lehetőségekről.
 - **Legolcsóbb elől:** Ez a beállítás a havi bérleti díj alapján **növekvő sorrendbe** állítja a találatokat. A legkedvezőbb árú ingatlan jelenik meg legelől, amit a magasabb árúak követnek.
 - **Legdrágább elől:** Az előző pont ellentéte, amely a bérleti díj szerint **csökkenő sorrendet** alkalmaz. A legmagasabb bérleti díjú, prémium kategóriás ingatlanok kerülnek a lista elejére.
- **Működése:** A rendezés módosítása azonnal újra szervezi a megjelenített hirdeteskártyákat, de a korábban beállított szűrőfeltételeket (pl. város, árkategória) nem módosítja.

Rendezés

Legújabb elől

Legújabb elől

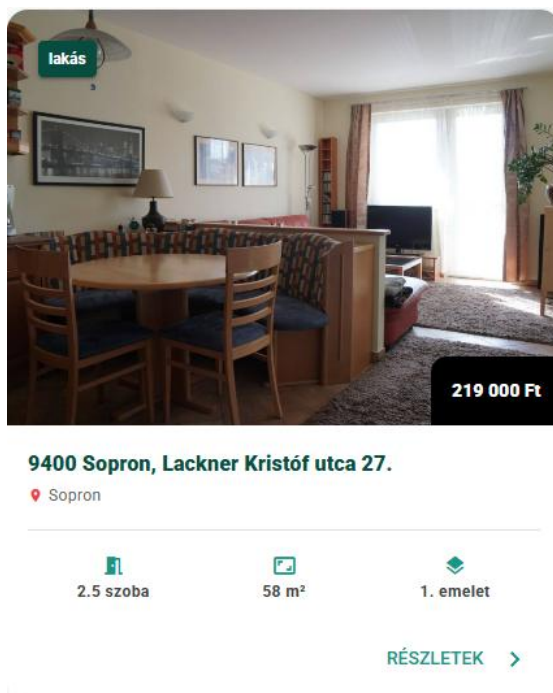
Legolcsóbb elől

Legdrágább elől

- A szűrőpanel jobb alsó sarkában két funkciógomb biztosítja a keresési folyamat vezérlését.
- **Keresés indítása gomb:**
 - **Funkció:** Erre a gombra kattintva a rendszer érvényesíti az összes korábban beállított szűrőfeltételt (ár, méret, város, kényelem stb.).
 - **Működés:** Az alkalmazás leszűri az adatbázist, és kizárólag azokat az ingatlanokat jeleníti meg a találati listában, amelyek minden megadott paraméternek maradéktalanul megfelelnek.
- **Szűrők törlése gomb:**
 - **Funkció:** Segítségével a felhasználó egyetlen kattintással eltávolíthatja az összes manuálisan megadott keresési feltételt.
 - **Működés:** A gomb lenyomása után a beviteli mezők és választólisták visszaállnak az alapértelmezett állapotukba, a rendszer pedig újra az összes elérhető albérletet listázza, az eredeti („Legújabb elől”) rendezési elv szerint.



- A keresési feltételeknek megfelelő találatok esztétikus, kártya alapú elrendezésben jelennek meg. A kártyák célja, hogy a felhasználó a legfontosabb információkat már az előnézetben, kattintás nélkül is megismerhesse.
 - **Ingatlan fotója:** Minden hirdetéshez tartozik egy reprezentatív kép, amely segíti a vizuális tájékozódást.
 - **Típusjelzés:** A kép bal felső sarkában egy címke jelzi az ingatlan típusát (például: *lakás*).
 - **Bérleti díj:** A kép jobb alsó sarkában, jól látható módon kerül feltüntetésre a havi bérleti díj forintban (Ft).
- A kártya információs blokkja strukturáltan közli az ingatlan specifikációit:
 - **Cím és város:** Az ingatlan pontos címe és a település neve jól elkülönítve látható.
 - **Műszaki adatok:** Három ikonnal segített adatmező mutatja be a lakás jellemzőit:
 - **Szobaszám:** Az ingatlanban található szobák mennyisége (pl. 2.5 szoba).
 - **Alapterület:** Az ingatlan mérete négyzetméterben (m²).
 - **Emelet:** Az ingatlan szintje (pl. 1. emelet).
- **Részletek gomb:** A kártya legalján elhelyezett interaktív hivatkozás lehetővé teszi az ingatlan teljes adatlapjának megnyitását. Itt a felhasználó elolvashatja a részletes leírást, megtekintheti a további fotókat, és felveheti a kapcsolatot a hirdetővel.



- A „Részletek” gombra kattintva az adott ingatlan teljes adatlapja nyílik meg. Ezen a felületen a látogatók részletesen tájékozódhatnak az albérlet minden paraméteréről és a hirdető elérhetőségeiről.

M&B lakhatás

← 1136 Budapest, Pannónia utca 15.

HAVI BÉRELETI DÍJ
1 400 000 Ft

Teljesítés / Megye: Budapest / Pest
 Szobák száma: 5,5 szoba
 Alapterület: 303 m²
 Ingatlan szintje: 2 szintes ház / Nincs lift
 Bútoroztatás: Bútoroztatott
 Várható kezdés: 2024. 09. 21.

Ingatlan leírása

Kiadó egy 303 nm-es, 5,5 szobás, 2 szintes ház 1 400 000 Ft/hó bérleti díjért. Az ingatlan bútorozott, lift nincs, tágas és exkluzív kialakításának köszönhetően ideális nagyobb családok vagy reprezentatív célra egyaránt.

Kapcsolattartó

Antal Jázmin
 Regisztráció
 +36 20 581 3334
 THE.JAZMIN@GMAIL.COM

M&B lakhatás
 © 2024. Minden jog fenntartva

Ügyfélszolgálat
 +36 20 300 3903

- **Visszalépés:** Az oldal tetején, a pontos cím mellett található egy „vissza” nyíl. Erre kattintva a felhasználó visszatérhet az előzőleg beállított szűrési találatokhoz az Albérlet Kereső oldalra.
- **Interaktív képgaléria:** Az ingatlanról készült fotók nagyméretű galériában tekinthetők meg. A képek között kettőféleképpen is lehet navigálni:
 - A kép két szélén található **irányító nyilakkal**.
 - A kép alatti **navigációs pontokra** (indikátorokra) kattintva.

← 1136 Budapest, Pannónia utca 15.

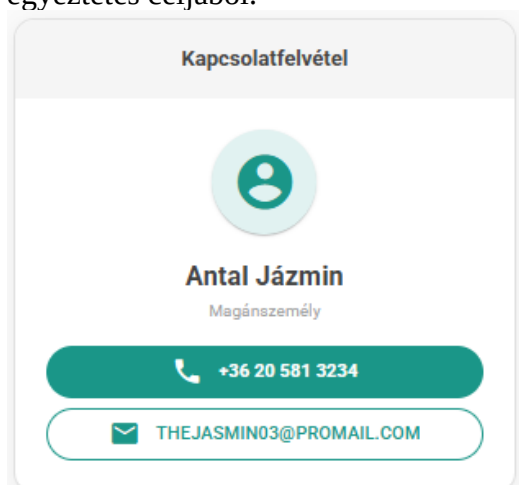
Ingatlan leírása

Kiadó egy 303 nm-es, 5,5 szobás, 2 szintes ház 1 400 000 Ft/hó bérleti díjért. Az ingatlan bútorozott, lift nincs, tágas és exkluzív kialakításának köszönhetően ideális nagyobb családok vagy reprezentatív célra egyaránt.

- Az oldal jobb oldali blokkja tartalmazza az összesített műszaki adatokat és a bérleti feltételeket. A felület logikus felépítése segíti a gyors információszerzést.
- **Pénzügyi adatok:** A blokk tetején kiemelt, zöld háttérrel jelenik meg a **Havi bérleti díj**, így ez a legmeghatározóbb információ azonnal látható.
- **Helyszín és méret:** Pontos település és vármegye megnevezés, a szobaszám (tizedes pontossággal), valamint az alapterület (m²) kerül felsorolásra.
- **Dinamikus adatmegjelenítés:** A rendszer az ingatlan típusa szerint változtatja a megjelenített információt:
 - **Lakás és Szoba esetén:** Megjelenik az **emelet** száma és a **lift** elérhetősége.
 - **Ház esetén:** Ezek helyett az épület **szintjeinek száma** látható.
- **Felszereltség és állapot:** A táblázat tartalmazza a **bútorozottságra** vonatkozó információt (pl. Bútorozott / Üres).
- **Időbeliség:** A lista alján a **Hirdetés feladva** mező jelzi a bejegyzés létrehozásának pontos dátumát, ami segít a hirdetés aktualitásának megítélésében.
- **Vizuális segédletek:** Minden adatsort egyedi ikon és szürke elválasztóvonal választ el a következőtől, ami javítja az olvashatóságot és a felhasználói élményt.

HAVI BÉRLETI DÍJ	
772 000 Ft	
	Település / Megye Budapest, Pest megye
	Szobák száma 5.0 szoba
	Alapterület 101 m²
	Emelet / Lift 1. emelet Nincs lift
	Bútorozottság Üres
	Hirdetés feladva 2026. 03. 23.

- Az ingatlan adatlapjának jobb alsó részén található a **Kapcsolatfelvétel** blokk, amely lehetővé teszi a bérlőjelöltek számára a hirdetővel való közvetlen kommunikációt.
- **Hirdetői profil:** A felület megjeleníti az ingatlan tulajdonosának nevét (például: *Antal Jázmin*).
- **Közvetlen interakció:** A rendszer a modern webes elvárásoknak megfelelően aktív linkekkel és gombokkal dolgozik:
 - **Telefonos elérhetőség:** A telefonszámra való kattintáskor a rendszer azonnal kezdeményezi a hívást a megadott számra.
 - **E-mail küldés:** Az e-mail cím megnyomásakor automatikusan megnyílik a felhasználó alapértelmezett levelezőprogramja, előkészítve az üzenetküldést a hirdető felé.
- **Felhasználói élmény:** Ez a megoldás minimalizálja az adminisztrációs lépéseket, így az érdeklődő egyetlen kattintással kapcsolatba léphet a tulajdonossal további egyeztetés céljából.



4.3. Hirdetés feladása

Hirdetés Feladása
Pár lépés és kikerült az ingatlanod az oldalra

Helyszín Ingatlan adatai Média & Leírás Kapcsolat

Megye Város

Teljes cím

FOLYTATÁS

M&B lakhatás
© 2026 Egyik jog sincs fenntartva

Ügyfélszolgálat
+36 20 500 3935

- **Elérhetőség:** A funkció a kezdőoldalon található **„Hirdetés feladása”** kártyára vagy gombra kattintva érhető el.
- **Visszatérés:** A navigációs konzisztencia itt is érvényesül: a bal felső sarokban található logóra vagy az „M&B lakhatás” felírra kattintva a felhasználó bármikor megszakíthatja a folyamatot és visszatérhet a főoldalra.

- A hirdetés rögzítése az ingatlan földrajzi elhelyezkedésének megadásával kezdődik. A rendszer egymásra épülő (dependens) mezőket használ, ami azt jelenti, hogy a kitöltés sorrendje kötött.
- **Vármegye kiválasztása:** A folyamat a megye megadásával indul.
 - A felhasználó egy legördülő menüből választhatja ki azokat a vármegyét, amelyek már szerepelnek a rendszer adatbázisában.
 - **Hibakezelés:** Amennyiben a mező üresen marad, a rendszer hibaüzenetet küld, és a következő (város) mező nem válik szerkeszthetővé.
- **Település megadása:** * A megye kiválasztása után válik aktívvá a város mező.
 - Itt is elérhető egy legördülő lista az adott megyéhez tartozó, már létező városokkal, de manuális adatbevitelre is van lehetőség.
 - **Hibakezelés:** Ha a felhasználó nem ad meg települést, a rendszer figyelmeztető jelzést ad.
- **Pontos cím rögzítése:** * A harmadik kötelező elem az ingatlan teljes címe.
 - **Felhasználói segédlet:** A mezőbe való kattintáskor egy „tipp” jelenik meg, amely útmutatást ad a cím helyes formátumára vonatkozóan.
 - **Hibakezelés:** A mező kitöltése kötelező, hiánya esetén hibaüzenet akadályozza meg a tovább lépést.

Hirdetés Feladása
Pár lépés és kikerül az ingatlanod az oldalra

Helyszín — Ingatlan adatai — Média & Leírás — Kapcsolat

Megye: Baranya X Város: Pécs X

Teljes cím

FOLYTATÁS

Hirdetés Feladása
Pár lépés és kikerül az ingatlanod az oldalra

Helyszín — Ingatlan adatai — Média & Leírás — Kapcsolat

Megye: Baranya ! Város: Pécs

Kötelező választani

Teljes cím: !

Kötelező mező

FOLYTATÁS

- Az adatok rögzítése után a rendszer ellenőrzi az adatbeviteli mezők tartalmát, mielőtt engedélyezné a tovább lépést.
- **Szakasz lezárása:** Amennyiben a felhasználó minden kötelező adatot (megye, város, pontos cím) megfelelően megadott, a **Folytatás** gomb aktívvá válik vagy megnyomásával a rendszer átlépteti a felhasználót a hirdetésfeladás következő, „Ingatlan adatok” szakaszára.
- **Szigorú hibaellenőrzés:** Ha bármelyik kötelezően kitöltendő mező üresen marad vagy érvénytelen adatot tartalmaz, az oldal egyértelmű **hibaüzenettel** tájékoztatja a felhasználót a hiányosságról.
- **Adatbiztonság:** Ez a logikai gát megakadályozza, hogy hiányos vagy félkész hirdetések kerüljenek mentésre az adatbázisba, biztosítva ezzel a hirdetési állomány minőségét.

- Ebben a szakaszban a felhasználó az ingatlan technikai jellemzőit és a bérleti feltételeket adhatja meg. A felület intelligens mezőket használ, amelyek az ingatlan típusa alapján változtatják működésüket.
- **Ingatlan típusa:** Egy lenyíló menü segítségével határozható meg, hogy **lakás**, **szoba** vagy **ház** kategóriába tartozik-e a hirdetés.
- **Bérleti díj:** Az ingatlan havi bérleti díja forintban (Ft/hó).
- **Validáció:** A rendszer biztonsági okokból minimum **10 000 Ft**-os díjat vár el; ennél alacsonyabb összeg esetén hibaüzenetet küld.
- **Alapterület:** Az ingatlan mérete négyzetméterben (m²).
 - **Validáció:** A minimálisan megadható érték **5 m²**. Amennyiben a bevitt adat ennél kisebb, hibaüzenet figyelmezteti a felhasználót.
- **Emelet megjelölése:** Itt adható meg, hányadik szinten található az albérlet.
 - **Speciális logika:** Ha a felhasználó **0**-át ír be, a rendszer intelligens módon a keresőoldalon automatikusan **„Földszint”** megnevezést jelenít meg.
- **Szobák száma:** Lehetőség van tört számok (pl. 2,5 szoba) megadására is.
 - **Kötöttség:** Amennyiben a típus korábban **„Szoba”** -ként lett meghatározva, ez a mező fix és nem módosítható.
- **Lift és Bútorozottság:** Egyszerű jelölőnégyzetek segítségével rögzíthető az ingatlan felszereltsége.
 - **Dinamikus tiltás:** Ha az ingatlan típusa **„Ház”** és az emelet mezőbe **0 vagy 1** került (tehát egyszintes az épület), a rendszer letiltja a lift opció módosítását, megelőzve a logikátlan adatbevitelt.
- **Visszalépés lehetősége:** A felületen található egy **„Vissza”** gomb, amely lehetővé teszi a helyszíni adatok ellenőrzését vagy javítását anélkül, hogy az aktuális adatok elvesznének.
- **Szakaszlezárás:** A **„Folytatás”** gomb megnyomásakor a rendszer ismételt validálást végez. Hiányos vagy helytelenül kitöltött mezők esetén a tovább lépés a következő („Média & Leírás”) szakaszra nem engedélyezett.

- Ebben a fázisban a felhasználó az ingatlan esztétikai bemutatására és a részletes tájékoztatásra koncentrálhat. A rendszer itt is szigorú formai követelményeket támaszt a minőségi hirdetések biztosítása érdekében.

- **Képek hozzáadása:** A „Képek kiválasztása (több is választható)” mezőre kattintva a felhasználó a saját eszközének fájlkezelőjéből töltheti fel az ingatlanról készült fotókat.
- **Többes kijelölés:** A rendszer támogatja egyszerre több fénykép kiválasztását, így segítve a gyorsabb adatfeltöltést és a galéria kialakítását.
- **Ellenőrzés:** A folyamat végén a rendszer validálja, hogy történt-e képfeltöltés, mivel fotók nélkül a hirdetés nem publikálható.

- **Ingatlan részletes leírása:** Ebben a szövegmezőben nyílik lehetőség az ingatlan egyedi előnyeinek, környékbeli adottságainak vagy speciális feltételeinek kifejtésére.
- **Karakterkorlátok és validáció:**
 - * **Minimum limit:** A leírásnak legalább **20 karakter** hosszúságúnak kell lennie, hogy elkerüljük a túl rövid, nem informatív hirdetéseket.
 - * **Maximum limit:** A rendszer legfeljebb **1000 karaktert** engedélyez, amit egy beépített számláló (0 / 1000) segít nyomon követni a mező jobb alsó sarkában.
 - * **Hibakezelés:** Amennyiben a szöveg hossza nem esik a meghatározott tartományba, a rendszer hibaüzenettel akadályozza meg a tovább lépést.

- **Rugalmas szerkesztés:** A „Vissza” gombbal a felhasználó bármikor visszatérhet az ingatlan paramétereire, ha menet közben jut eszébe egy javítandó adat.
- **Szakaszlezárás:** A „Folytatás” gomb csak akkor válik érvényessé, ha a képek feltöltése és a leírás validációja is sikeresen megtörtént.

VISSZA FOLYTATÁS

Hirdetés Feladása

Pár lépés és kikerül az ingatlanod az oldalra

✓ Helyszín
✓ Ingatlan adatok
✓ Média & Leírás
Kapcsolat

Képek kiválasztása (több is választható)

sopron_03_01.png sopron_03_02.png sopron_03_03.png sopron_03_04.png sopron_03_05.png

Az ingatlan részletes bemutatása...

12345

Legalább 20 karakter szükséges 5 / 1000

VISSZA FOLYTATÁS

M&B lakhatás
6 Egyik jog sincs fenntartva

Ügyfélszolgálat
+36 20 500 3923

Kérjük, töltsön ki minden mezőt megfelelően!

- A hirdetés aktiválása előtt a rendszer bekéri a bérbeadó közvetlen elérhetőségeit. A felület kiemelt figyelmet fordít az adatok formai helyességére, hogy megelőzze a sikertelen kapcsolatfelvételi kísérleteket.

Hirdetés Feladása

Pár lépés és kikerül az ingatlanod az oldalra

✓ Helyszín
✓ Ingatlan adatok
✓ Média & Leírás
✓ Kapcsolat

E-mail cím

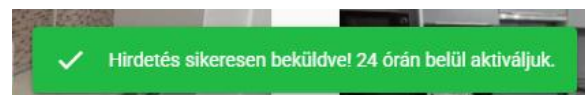
Teljes név

Telefonszám
+36

VISSZA HIRDETÉS BEKÜLDÉSE

- **E-mail cím:** A hirdető elektronikus levelezési címe.
- **Validáció:** A rendszer ellenőrzi a megadott e-mail formátumát (pl. tartalmaz-e @ jelet és pontot). Üres mező vagy helytelen formátum esetén hibaüzenet jelenik meg.
- **Teljes név:** A tulajdonos vagy kapcsolattartó neve (pl. Kovács Sándor).
- **Validáció:** A mező kitöltése kötelező; hiánya esetén a rendszer figyelmeztetést küld.
- **Telefonszám:** A közvetlen telefonos elérhetőség megadása.
- **Formai kötöttség:** A mező maszkolt adatbevitelt használ, így kizárólag számok adhatóak meg a szabványos magyar telefonszám formátumban (+36 00 000 0000).

- **Adatok ellenőrzése:** A felhasználónak ezen a ponton is lehetősége van a „Vissza” gombbal az összes korábbi lépés (Helyszín, Ingatlan adatok, Média) felülvizsgálatára.
- **Beküldés:** Amennyiben minden mező megfelel a validációs szabályoknak, a „Hirdetés beküldése” gombbal a folyamat lezárul.
- **Adminisztrátori jóváhagyás:** A beküldött adatok nem válnak azonnal láthatóvá. A hirdetés egy **ellenőrzési folyamaton** megy keresztül, ahol az adminisztrátor jóváhagyása szükséges a publikáláshoz.
 - **Célja:** Ez a biztonsági lépés garantálja, hogy a platformon ne jelenhessenek meg hamis, félrevezető vagy irreleváns („troll”) hirdetések.



5. Tesztelés

Teszteset:	TC-001	Eredmény:	PASS
API-végpont	GET /api/alberletek		
Metódus	test_alberletek_index_json_struktura()		
Cél	Az albérletek listázásának és a válasz JSON szerkezetének ellenőrzése.		

Teszteset:	TC-002	Eredmény:	PASS
API-végpont	GET api/alberletek?min_ar=9999999		
Metódus	test_ar_szures_mukodik()		
Cél	Az ár alapú szűrés működésének ellenőrzése (extrém magas árra ne legyen találat).		

Teszteset:	TC-003	Eredmény:	PASS
API-végpont	GET /api/users/check		
Metódus	test_tulajdonos_ellenorzes_letezo_es_nem_letezo_emailre()		
Cél	Annak ellenőrzése, hogy a rendszer helyesen jelzi-e, ha egy email cím még nem létezik vagy hibás.		

Teszteset:	TC-004	Eredmény:	PASS
API-végpont	POST /api/alberletek		
Metódus	test_store_elutasitja_a_tul_rovid_leirast()		
Cél	A szerveroldali validáció tesztelése: a 20 karakternél rövidebb leírást el kell utasítania (422 hiba).		

Teszt eset:	TC-005	Eredmény:	PASS
API-végpont	GET /api/varosok?filtered=true		
Metódus	test_varosok_listazasa()		
Cél	A városok listájának és a lenyíló menükhöz szükséges (value/label) formátumnak az ellenőrzése.		

Teszt eset:	TC-006	Eredmény:	PASS
API-végpont	GET /api/megyek		
Metódus	test_megyek_listazasa_mukodik()		
Cél	A megyék listázásának és a JSON struktúra (value/label) ellenőrzése.		

Teszt eset:	TC-007	Eredmény:	PASS
API-végpont	GET /api/alberletek/{id}		
Metódus	test_alberlet_torlese_mukodik()		
Cél	Egy létező hirdetés törlésének ellenőrzése az adatbázisból az ID alapján.		

6. Fejlesztési lehetőségek

1. "Kedvencek" funkció (LocalStorage vagy Auth)

A felhasználók imádják jelölgetni, ami tetszik nekik.

- **A fejlesztés:** Egy kis szív ikon a hirdetésekre.
- **Pro megoldás:** Ha nem akarsz beléptetést a sima usereknek, használd a böngésző **LocalStorage**-át. Így, ha visszatér az oldalra, még mindig ott lesznek a kedvencei anélkül, hogy adatbázisba mentenél bármit róluk.

2. Térképes nézet (Leaflet.js vagy Google Maps)

- Egy albérletnél a környék a legfontosabb.

- **A fejlesztés:** Mivel a **Varos** és a **Cim** megvan, használhatsz egy ingyenes "geocoding" szolgáltatást, ami a címből koordinátákat csinál. A frontend pedig egy térképen pöttyökként jelenítené meg a lakásokat.

3. Az albérlet adatok törlése

- **Jelenlegi állapot:** Az albérlet törlésekor a kapcsolódó rekordok eltávolítása megtörténik, de a képek és fájlhivatkozások az adatbázisban maradnak.
- **Feladat:** A törlési logika kiegészítése a fájlrendszer-takarítással és a kapcsolódó képtáblák rekordjainak kényszerített törlésével.
- **Cél:** Teljes körű adatintegritás és automatikus erőforrás-felszabadítás.

Jogi tájékoztatás

A projektben felhasznált képek és egyéb médiaelemek oktatási céllal, kizárólag zárt iskolai környezetben kerültek felhasználásra, nem üzletszerű módon. A projekt nem kerül nyilvánosságra vagy internetes közzétételre. A szerzői jogról szóló, 1999. évi LXXVI. törvény 35. § alapján az idézett vagy átvett tartalmaknál, a felhasznált képeknél törekedtünk a forrásmegjelölésre és az oktatási cél által indokolt mértékű felhasználásra.

Nagykanizsa, 2026. április 27.

Cséfalvay Bálint

Horváth Márk Attila

Irodalomjegyzék

HTML	https://developer.mozilla.org/en-US/docs/Web/HTML
CSS	https://developer.mozilla.org/en-US/docs/Web/CSS https://www.w3schools.com/
Javascript	https://developer.mozilla.org/en-US/docs/Web/JavaScript
Vue.js fejlesztői környezet	https://vuejs.org/
Vue Router	https://router.vuejs.org/
Pinia Store	https://pinia.vuejs.org/
Quasar Framework	https://quasar.dev/
Visual Studio Code	https://code.visualstudio.com/docs
Trello	https://trello.com/hu/tour
MVVM	https://en.wikipedia.org/wiki/Model-view-viewmodel
MySQL	https://www.w3schools.com/mysql/default.asp